MC458 - Projeto e Análise de Algoritmos I

Prof. Dr. Hilder Vitor Lima Pereira

Instituto de computação

UNICAMP

17 de março de 2026

1 Referência sugerida

2 Divisão e conquista
- Divisão e conquista
- Exponenciação rápida
- Ordenação: MergeSort
- Subvetor com soma máxima

3 Perguntas ou comentários?

Referência sugerida

Referência sugerida

CLRS Introduction to Algorithms. Cormen, Leiserson, Rivest e Stein.

Divisão e conquista

Divisão e conquista:

Divisão e conquista

Divisão e conquista

A estratégia de divisão e conquista

1. Divisão: dividir o problema em subproblemas semelhantes, mas com instâncias menores
2. Conquista: cada subproblema é resolvido recursivamente, ou diretamente se os subproblemas forem pequenos
3. Combinação: as soluções dos subproblemas são combinadas para obter uma solução da instância original

Projeto de algoritmos por divisão e conquista

Algumas características

- geralmente, criamos pelo menos duas subinstâncias
- geralmente, elas são muito menores que a instância original
- o custo de combinar as soluções é baixo

Algoritmo genérico

```
DivisaoConquista(x)
if a instância x é suficientemente pequena then
    return Solucao(x)
else
    decomponha x em instâncias menores x_1, x_2, ··· , x_k
    for 1 ≤ i ≤ k do
        y_i ← DivisaoConquista(x_i)
    combine as soluções y_i para obter uma solução y
    return y
```

Divisão e conquista:

Exponenciação rápida

Exemplo: Exponenciação com divisão e conquista

Problema

Entrada:

- dois inteiros a, n com $n \geq 0$

Saída:

- o inteiro a^n

Exemplo: Exponenciação com divisão e conquista

Problema

Entrada:

- dois inteiros a, n com $n \geq 0$

Saída:

- o inteiro a^n

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?

- a^{n-1}

Exemplo: Exponenciação com divisão e conquista

Problema

Entrada:

- dois inteiros a, n com $n \geq 0$

Saída:

- o inteiro a^n

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?

- a^{n-1}
- $a^{n/2}$

Hipótese de indução

Para um inteiro $n \geq 1$, sabemos calcular a^m se $0 \leq m < n$.

Exemplo: Exponenciação com divisão e conquista

Problema

Entrada:

- dois inteiros a, n com $n \geq 0$

Saída:

- o inteiro a^n

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?

- a^{n-1}
- $a^{n/2}$

Hipótese de indução

Para um inteiro $n \geq 1$, sabemos calcular a^m se $0 \leq m < n$.

Que algoritmo conseguimos projetar assim?

Exponenciação com divisão e conquista

1. Divisão: $a^n = (a^{n/2})^2$
2. Conquista: calculamos $y = a^{\lfloor n/2 \rfloor}$ recursivamente
3. Combinação: calculamos y^2 e ajustamos o resultado final dependendo da paridade de a

Exponenciação rápida

$\texttt{Pow}(a, n)$

Input: $a \in \mathbb{Z}$, $n \in \mathbb{N}$

Output: $a^n \in \mathbb{Z}$

```
if n = 0 then
    return 1
y ← Pow(a, ⌊n/2⌋)
if n mod 2 = 0 then
    x ← y · y
else
    x ← x · y · y
return x
```

Complexidade temporal de $\texttt{Pow}(a, n)$

Vamos contar o número de multiplicações (note que **não** são operações elementares...)

$$T(n) \leq \begin{cases} 0, & n = 0 \\ T(\lfloor n/2 \rfloor) + 2 & n > 0 \end{cases}$$

Complexidade temporal de $\texttt{Pow}(a, n)$

Vamos contar o número de multiplicações (note que **não** são operações elementares...)

$$T(n) \leq \begin{cases} 0, & n = 0 \\ T(\lfloor n/2 \rfloor) + 2 & n > 0 \end{cases}$$

Usando o método da iteração e substituição, vemos que $T(n) \in O(\log n)$. Note que isso é muito melhor do que o algoritmo ingênuo, que é linear.

Comentário sobre exponenciação rápida

Consideramos o problema da exponenciação, cuja entrada é $(a, n) \in \mathbb{Z} \times \mathbb{N}$ e a saída é a^n.
Vimos dois algoritmos:

- Trivial: subproblema: a^{n-1}; número de multiplicações $T(n) \in O(n)$.
- Rápido: subproblema: $a^{\lfloor n/2 \rfloor}$; número de multiplicações $R(n) \in O(\log n)$.

Comentário sobre exponenciação rápida

Consideramos o problema da exponenciação, cuja entrada é $(a, n) \in \mathbb{Z} \times \mathbb{N}$ e a saída é a^n.
Vimos dois algoritmos:

- Trivial: subproblema: a^{n-1}; número de multiplicações $T(n) \in O(n)$.
- Rápido: subproblema: $a^{\lfloor n/2 \rfloor}$; número de multiplicações $R(n) \in O(\log n)$.

Esses algoritmos são lineares, quadráticos, cúbicos, exponenciais, etc?

Comentário sobre exponenciação rápida

Consideramos o problema da exponenciação, cuja entrada é $(a, n) \in \mathbb{Z} \times \mathbb{N}$ e a saída é a^n.
Vimos dois algoritmos:

- Trivial: subproblema: a^{n-1}; número de multiplicações $T(n) \in O(n)$.
- Rápido: subproblema: $a^{\lfloor n/2 \rfloor}$; número de multiplicações $R(n) \in O(\log n)$.

Esses algoritmos são lineares, quadráticos, cúbicos, exponenciais, etc?
Qual seria a complexidade em termos de operações básicas?

Divisão e conquista:

Ordenação: MergeSort

Exemplo: ordenação

Até agora:

- ordenamos incrementalmente com o Insertion-Sort
- vimos que sua complexidade de pior caso é $\Theta(n^2)$
- Insertion-Sort, Selection-Sort, Bubble-Sort e outros algoritmos de ordenação quadráticos se baseiam em subproblemas de tamanho $n - 1$

Exemplo: ordenação

Até agora:

- ordenamos incrementalmente com o Insertion-Sort
- vimos que sua complexidade de pior caso é $\Theta(n^2)$
- Insertion-Sort, Selection-Sort, Bubble-Sort e outros algoritmos de ordenação quadráticos se baseiam em subproblemas de tamanho $n - 1$

E se usarmos subproblemas muito menores, como de tamanho $\lfloor n/2 \rfloor$?

Exemplo: ordenando usando divisão e conquista

`MergeSort` é um exemplo clássico de divisão e conquista.

Ideia:

1. Divisão: divida um vetor de tamanho n em dois subvetores de tamanhos $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$
2. Conquista: ordene os dois subvetores recursivamente
3. Combinação: intercale os dois subvetores obtendo um vetor ordenado

Trabalhamos com subvetores $v[i...f]$, ou seja, i marca a primeira e f a última posição.
Na primeira chamada, usamos $i = 0$ e $f = n - 1$.

`MergeSort`(v, i, f)

Input: $i, f \in \mathbb{N}$ e vetor $v \in \mathbb{Z}^{f-i+1}$

```
if f > i then
    m ← ⌊(i + f)/2⌋
    MergeSort(v, i, m)
    MergeSort(v, m + 1, f)
    Intercala(v, i, m, f)
```

---

	i				m				f
v	66	33	55	44	99	11	77	22	88

Trabalhamos com subvetores $v[i...f]$, ou seja, i marca a primeira e f a última posição.
Na primeira chamada, usamos $i = 0$ e $f = n - 1$.

```
MergeSort(v, i, f)
Input: i, f ∈ ℕ e vetor v ∈ ℤ^{f−i+1}
if f > i then
    m ← ⌊(i + f)/2⌋
    MergeSort(v, i, m)
    MergeSort(v, m + 1, f)
    Intercala(v, i, m, f)
```

	i				m				f
v	33	44	55	66	99	11	77	22	88

Trabalhamos com subvetores $v[i...f]$, ou seja, i marca a primeira e f a última posição.
Na primeira chamada, usamos $i = 0$ e $f = n - 1$.

`MergeSort`(v, i, f)

Input: $i, f \in \mathbb{N}$ e vetor $v \in \mathbb{Z}^{f-i+1}$

```
if f > i then
    m ← ⌊(i + f)/2⌋
    MergeSort(v, i, m)
    MergeSort(v, m + 1, f)
    Intercala(v, i, m, f)
```

---

	i				m				f
v	33	44	55	66	99	11	22	77	88

Trabalhamos com subvetores $v[i...f]$, ou seja, i marca a primeira e f a última posição.
Na primeira chamada, usamos $i = 0$ e $f = n - 1$.

`MergeSort`(v, i, f)

Input: $i, f \in \mathbb{N}$ e vetor $v \in \mathbb{Z}^{f-i+1}$

```
if f > i then
    m ← ⌊(i + f)/2⌋
    MergeSort(v, i, m)
    MergeSort(v, m + 1, f)
    Intercala(v, i, m, f)
```

---

	i				m				f
v	11	22	33	44	55	66	77	88	99

Combinando soluções dos subproblemas

Problema de intercalar dois subvetores

Entrada:

- inteiros i, m, f
- vetor $v[i \ldots f]$ tal que
  1. subvetor $v[i \ldots m]$ está ordenado
  2. subvetor $v[m+1 \ldots f]$ está ordenado

Saída:

- rearranjo $v[i \ldots f]$ ordenado

Pseudocódigo de Intercala

```
Intercala(v, i, m, f)
Input: i, m, f ∈ ℕ e v ∈ ℤ^{f−i+1}
for (k ← i; k ≤ f; k++) do
    T[k] ← v[k]
(k, e, d) ← (i, f, m + 1)
while e ≤ m and d ≤ f do
    if T[e] < T[d] then
        v[k] ← T[e]
        (e, k) ← (e + 1, k + 1)
    else
        v[k] ← T[d]
        (d, k) ← (d + 1, k + 1)
while e ≤ m do
    v[k] ← T[e]
    (e, k) ← (e + 1, k + 1)
while d ≤ f do
    v[k] ← T[d]
    (d, k) ← (d + 1, k + 1)
```

Complexidade de Intercala

Tamanho da entrada: $n = i - f + 1$

Complexidade de Intercala

Tamanho da entrada: $n = i - f + 1$

- Cada iteração de cada laço tem $O(1)$ operações básicas
- O primeiro laço tem n iterações
- Cada outro laço tem no máximo n iterações
- Então o número de operações é $\leq 4 \cdot c \cdot n$
- Conclusão: Consumo de tempo: $\Theta(n)$

Complexidade de Intercala

Tamanho da entrada: $n = i - f + 1$

- Cada iteração de cada laço tem $O(1)$ operações básicas
- O primeiro laço tem n iterações
- Cada outro laço tem no máximo n iterações
- Então o número de operações é $\leq 4 \cdot c \cdot n$
- Conclusão: Consumo de tempo: $\Theta(n)$

Na verdade, podemos mostrar que os três últimos laços **juntos** têm n iterações, mas isso só mudaria o custo de $4 \cdot c \cdot n$ para $2 \cdot c \cdot n$, o que ainda é $\Theta(n)$...

Complexidade de MergeSort

```
MergeSort(v, i, f)
Input: i, f ∈ ℕ e vetor v ∈ ℤ^{f−i+1}
if f > i then
    m ← ⌊(i + f)/2⌋
    MergeSort(v, i, m)
    MergeSort(v, m + 1, f)
    Intercala(v, i, m, f)
```

- Tamanho da entrada: $n = f - i + 1$
- Seja $T(n)$ o número de instruções executadas no pior caso

$$T(n) = \begin{cases} c & \text{se } n \leq 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \hat{c} \cdot n & \text{se } n \geq 2 \end{cases}$$

Complexidade de MergeSort

```
MergeSort(v, i, f)
Input: i, f ∈ ℕ e vetor v ∈ ℤ^{f−i+1}
if f > i then
    m ← ⌊(i + f)/2⌋
    MergeSort(v, i, m)
    MergeSort(v, m + 1, f)
    Intercala(v, i, m, f)
```

- Tamanho da entrada: $n = f - i + 1$
- Seja $T(n)$ o número de instruções executadas no pior caso

$$T(n) = \begin{cases} c & \text{se } n \leq 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \hat{c} \cdot n & \text{se } n \geq 2 \end{cases}$$

Neste ponto do curso, já somos capazes de perceber e provar que $T(n) \in \Theta(n \log n)$

Discussão

A estratégia de divisão e conquista nos permitiu ordenar em tempo $O(n \log n)$ em vez de $O(n^2)$

Conseguimos isso porque

1. a etapa de combinação era muito mais barata (linear) do que a ordenação que já conhecíamos $O(n^2)$

Discussão

A estratégia de divisão e conquista nos permitiu ordenar em tempo $O(n \log n)$ em vez de $O(n^2)$

Conseguimos isso porque

1. a etapa de combinação era muito mais barata (linear) do que a ordenação que já conhecíamos $O(n^2)$
2. ordenar $n/2$ elementos em tempo $n^2/4$ é uma melhora de 4 vezes, mas só precisamos de duas chamadas recursivas, então, ainda ganhamos um fator $2\times$

Divisão e conquista:

Subvetor com soma máxima

Subvetor com soma máxima

Dado um subvetor $v[i, ..., j]$, defina sua soma como

$$\texttt{soma}(v, i, j) = v[i] + v[i+1] + \cdots + v[j-1] + v[j]$$

Subvetor com soma máxima

Dado um subvetor $v[i, ..., j]$, defina sua soma como

$$\texttt{soma}(v, i, j) = v[i] + v[i+1] + \cdots + v[j-1] + v[j]$$

Problema

- Entrada: $v \in \mathbb{Z}^n$ com $n \geq 1$
- Saída: $i, j \in \mathbb{N}$ tal que $\texttt{soma}(v, i, j)$ é máxima

Ou seja, $\forall (k, \ell) \in \mathbb{N}^2\, \texttt{soma}(v, k, \ell) \leq \texttt{soma}(v, i, j)$

Subvetor com soma máxima

Dado um subvetor $v[i, ..., j]$, defina sua soma como

$$\texttt{soma}(v, i, j) = v[i] + v[i+1] + \cdots + v[j-1] + v[j]$$

Problema

- Entrada: $v \in \mathbb{Z}^n$ com $n \geq 1$
- Saída: $i, j \in \mathbb{N}$ tal que $\texttt{soma}(v, i, j)$ é máxima

Ou seja, $\forall (k, \ell) \in \mathbb{N}^2\, \texttt{soma}(v, k, \ell) \leq \texttt{soma}(v, i, j)$

Como podemos resolver esse problema?

Subvetor com soma máxima: algoritmo trivial

- Crie uma variável m para armazenar o valor máximo
- Para cada (i, j), calcule $\texttt{soma}(v, i, j)$
- Atualize m e guarde i, j se $\texttt{soma}(v, i, j) > m$

Subvetor com soma máxima: algoritmo trivial

- Crie uma variável m para armazenar o valor máximo
- Para cada (i, j), calcule $\texttt{soma}(v, i, j)$
- Atualize m e guarde i, j se $\texttt{soma}(v, i, j) > m$

Quantas operações o algoritmo faz?

- Há $\Theta(n^2)$ pares (i, j)
- Cada chamada $\texttt{soma}(v, i, j)$ efetua $O(n)$ operações <u>aritiméticas</u>
- Portanto, $T(n) \in O(n^3)$

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n - 1$

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n - 1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n - 1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n - 1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:
  - A soma máxima está no vetor $v[0, ..., n - 2]$ (e seu valor é s)

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n - 1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:
  - A soma máxima está no vetor $v[0, ..., n - 2]$ (e seu valor é s)
  - Ou a soma máxima contém $v[n - 1]$

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n-1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:
  - A soma máxima está no vetor $v[0, ..., n-2]$ (e seu valor é s)
  - Ou a soma máxima contém $v[n-1]$
  - Para distinguir os dois casos, calculamos a soma máxima a partir da última posição e comparamos com s

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n-1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:
  - A soma máxima está no vetor $v[0, ..., n-2]$ (e seu valor é s)
  - Ou a soma máxima contém $v[n-1]$
  - Para distinguir os dois casos, calculamos a soma máxima a partir da última posição e comparamos com s
  - Podemos fazer isso em tempo $\Theta(n)$

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n-1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:
  - A soma máxima está no vetor $v[0, ..., n-2]$ (e seu valor é s)
  - Ou a soma máxima contém $v[n-1]$
  - Para distinguir os dois casos, calculamos a soma máxima a partir da última posição e comparamos com s
  - Podemos fazer isso em tempo $\Theta(n)$
- Assim, temos um algoritmo com a seguinte complexidade temporal (em termos de operações aritméticas)

$$T(n) = T(n-1) + c \cdot n$$

Subvetor com soma máxima: algoritmo recursivo básico

- Podemos assumir que sabemos resolver o problema para vetores de tamanho $n-1$
- Usamos recursão para obter (i, j, s), onde $s = \texttt{soma}(v, i, j)$.
- Agora, temos dois casos:
  - A soma máxima está no vetor $v[0, ..., n-2]$ (e seu valor é s)
  - Ou a soma máxima contém $v[n-1]$
  - Para distinguir os dois casos, calculamos a soma máxima a partir da última posição e comparamos com s
  - Podemos fazer isso em tempo $\Theta(n)$
- Assim, temos um algoritmo com a seguinte complexidade temporal (em termos de operações aritméticas)

$$T(n) = T(n-1) + c \cdot n$$

- Logo, um algoritmo quadrático, pois $T(n) \in O(n^2)$

Subvetor com soma máxima: divisão e conquista

Como podemos usar divisão e conquista nesse problema?

Subvetor com soma máxima: divisão e conquista

Como podemos usar divisão e conquista nesse problema?

- Assuma que você sabe resolver esse problema para vetores de tamanho menor que n (indução forte)
- Então, sabemos achar (i_e, j_e) para a soma máxima de $v[0, ..., n/2]$
- E sabemos achar (i_d, j_d) para a soma máxima de $v[n/2 + 1, ..., n]$
- Essa foi a divisão. Como fica a etapa de conquista?

Subvetor com soma máxima: divisão e conquista

Como podemos usar divisão e conquista nesse problema?

- Assuma que você sabe resolver esse problema para vetores de tamanho menor que n (indução forte)
- Então, sabemos achar (i_e, j_e) para a soma máxima de $v[0, ..., n/2]$
- E sabemos achar (i_d, j_d) para a soma máxima de $v[n/2 + 1, ..., n]$
- Essa foi a divisão. Como fica a etapa de conquista?
- Temos dois casos para um subvetor $v[i, ..., j]$ com soma máxima
  1. Ele não contém $v[n/2]$
     - então, solução é (i_e, j_e) ou (i_d, j_d)
  2. Ele contém $v[n/2]$
     - então, podemos calcular solução em tempo $O(n)$

Subvetor com soma máxima: divisão e conquista

Exercício: Escreva um algoritmo `subVetorMaximoMeio`(v, i, j) que encontre subvetor de soma máxima contendo $v[n/2]$ e rode em tempo $O(n)$.

Subvetor com soma máxima

$\texttt{subVetorMaximo}(v, i, f)$

Input: $I, F \in \mathbb{N}$ e vetor $v \in \mathbb{Z}^{F-I+1}$
Output: $(i, j, s) \in \mathbb{Z} : \texttt{soma}(v, i, j) = s$ é máxima

```
n ← f − i + 1
if n = 1 then
    return (I, F, v[I])
m ← ⌊(I + F)/2⌋
(i_e, j_e, s_e) ← subVetorMaximo(v, I, m)
(i_d, j_d, s_d) ← subVetorMaximo(v, m + 1, D)
(i_m, j_m, s_m) ← subVetorMaximoMeio(v, I, F)
if s_e ≥ s_d and s_e ≥ s_m then
    return (i_e, j_e, s_e)
if s_d ≥ s_e and s_d ≥ s_m then
    return (i_d, j_d, s_d)
return (i_m, j_m, s_m)
```

Subvetor com soma máxima: complexidade

O número de operações aritméticas (<u>não</u> básicas) efetuadas por esse algoritmo é

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + an + b$$

onde a e b são constantes.

Assim como para o merge-sort, temos

$$T(n) \in O(n \log n)$$

Subvetor com soma máxima: complexidade

O número de operações aritméticas (<u>não</u> básicas) efetuadas por esse algoritmo é

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + an + b$$

onde a e b são constantes.

Assim como para o merge-sort, temos

$$T(n) \in O(n \log n)$$

Note que divisão e conquista melhorou nossa solução de $O(n^3)$ para $O(n \log n)$

Perguntas ou comentários?